



APOSTILA DE COMPILADORES

Aula 8 — Análise Sintática Ascendente

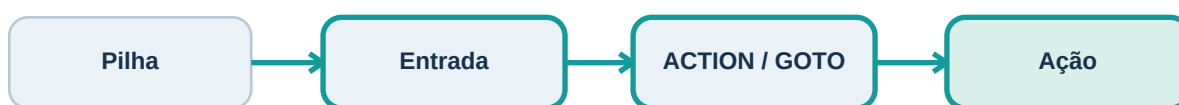
Do reconhecimento de estruturas à construção de parsers LR para a
linguagem MINIC

Material de apoio • sequência após a Aula 7

1. Objetivos de aprendizagem

Ao final desta aula, você deverá compreender como um analisador sintático ascendente reconstrói uma derivação a partir dos tokens, distinguir as ações **shift**, **reduce**, **accept** e **error**, e interpretar a organização de um parser LR aplicado à MINIC.

- Relacionar análise descendente e ascendente.
- Executar manualmente uma sequência shift-reduce.
- Entender itens LR(0), estados e transições.
- Ler uma tabela ACTION/GOTO e identificar conflitos.
- Preparar a implementação do parser da Etapa 2 do projeto MINIC.



O parser consulta a tabela para decidir deslocar, reduzir, aceitar ou sinalizar erro.

Figura 1 — Visão esquemática do ciclo de decisão de um parser ascendente. Ilustração didática produzida para esta apostila.

Conexão com a aula anterior. Na Aula 7, a derivação era construída a partir do símbolo inicial. Agora, o parser parte da entrada e tenta reduzi-la até chegar ao símbolo inicial.

2. O que é análise sintática ascendente?

Na análise ascendente, o parser começa pelos tokens da entrada e combina fragmentos reconhecidos até formar estruturas maiores. O objetivo é produzir uma sequência de reduções que corresponda a uma derivação mais à direita invertida.

A estratégia é especialmente importante para gramáticas de expressões, comandos e chamadas de função, pois permite lidar de forma sistemática com precedência, associatividade e estruturas recursivas.

Exemplo de gramática:

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

Para a entrada **id + id * id**, o parser não escolhe arbitrariamente qualquer redução: a tabela determina quando deslocar o próximo token e quando reduzir um trecho já reconhecido.

Em termos práticos, o parser mantém uma pilha de estados e símbolos, consulta o próximo token e realiza uma ação.

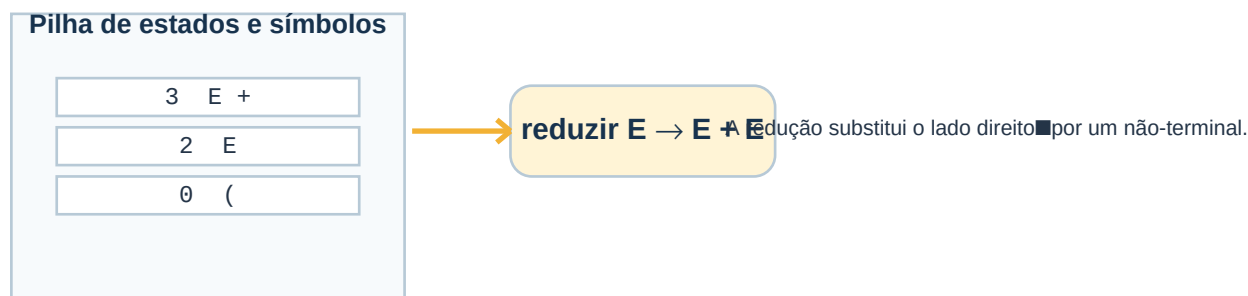


Figura 2 — Pilha e decisão de redução em uma análise shift-reduce. Ilustração didática.

3. As quatro ações do parser

Ação	Significado	Exemplo
shift s5	Desloca o token de entrada e vai ao estado 5.	Ler id e empilhá-lo.
reduce $A \rightarrow \beta$	Retira β da pilha e empilha A, consultando GOTO.	Reduz id para F.
accept	A entrada foi reconhecida integralmente.	Chegou a \$ e a produção inicial foi concluída.
error	Nenhuma ação válida para o estado e o token atuais.	Token inesperado ou estrutura incompleta.

Importante: uma redução não “apaga” a informação. Ela transforma uma sequência de símbolos em um não-terminal, preservando a estrutura necessária para continuar o reconhecimento.

Exercício guiado. Considere a produção $F \rightarrow id$. Ao ler o token **id**, o parser pode deslocá-lo. Quando a tabela indicar redução por $F \rightarrow id$, ele remove o par associado ao token e empilha F. O estado seguinte é obtido por GOTO do estado que ficou no topo da pilha.

A combinação entre a pilha e as tabelas é o que torna o parser determinístico: a gramática e o estado atual definem a próxima ação.

4. Exemplo completo de shift-reduce

Use a gramática reduzida abaixo para acompanhar a entrada **id + id**:

```
E' → E
E → E + T | T
T → id
```

Pilha	Entrada	Ação
0	id + id \$	shift 2
0 id 2	+ id \$	reduce T → id
0 T 1	+ id \$	reduce E → T
0 E 3	+ id \$	shift 4
0 E 3 + 4	id \$	shift 2
0 E 3 + 4 id 2	\$	reduce T → id
0 E 3 + 4 T 5	\$	reduce E → E + T
0 E 3	\$	accept

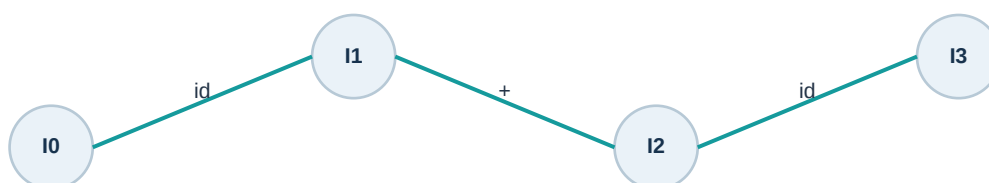
Observe que o parser reconhece primeiro os operandos, depois forma **T**, em seguida **E** e finalmente reduz a soma. A sequência de reduções é a derivação mais à direita lida de trás para frente.

5. Itens LR(0) e construção de estados

Um item LR(0) é uma produção com um ponto que marca quanto da produção já foi reconhecido. Por exemplo:

```
E → E • + T
E → E + • T
E → E + T •
```

O ponto evolui quando o parser desloca símbolos. Quando chega ao final da produção, o item indica uma possível redução.



Itens LR(0) representam posições possíveis do parser dentro das produções.

Figura 3 — Esquema conceitual de estados e transições de um autômato LR. Ilustração didática baseada nos conceitos de itens e fechamento.

Fechamento: se um item possui o ponto antes de um não-terminal, as produções desse não-terminal entram no conjunto de itens. **GOTO:** desloca o ponto sobre um símbolo e calcula o próximo estado.

O conjunto canônico de itens fornece a base para construir as tabelas ACTION e GOTO. Em ferramentas reais, essa construção é automatizada; no projeto, é importante compreender o resultado para diagnosticar conflitos e erros.

6. Tabelas ACTION e GOTO

A tabela **ACTION** é consultada para terminais e informa shift, reduce, accept ou error. A tabela **GOTO** é consultada para não-terminais depois de uma redução.

Estado	id	+	\$	E	T
0	s2	—	—	1	3
1	—	s4	acc	—	—
2	—	r3	r3	—	—
3	—	r2	r2	—	—
4	s2	—	—	—	5
5	—	r1	r1	—	—

Legenda: **sN** = shift para o estado N; **rN** = reduzir pela produção N; **acc** = aceitar. O traço representa erro ou célula não definida.

Conflito shift/reduce: a mesma célula pede deslocamento e redução. **Conflito reduce/reduce:** duas reduções são possíveis. Precedência e associatividade podem resolver alguns conflitos de expressões; outros revelam uma gramática ambígua ou inadequada.

7. SLR(1), LR(1) e escolha da técnica

As famílias LR diferem principalmente na quantidade de contexto usada para decidir reduções.

Técnica	Característica	Uso didático
LR(0)	Itens sem lookahead; simples, porém limitado.	Compreender estados e o mecanismo básico.
SLR(1)	Usa FOLLOW para restringir reduções.	Boa ponte entre teoria e implementação.
LR(1)	Itens carregam lookaheads específicos.	Maior poder de reconhecimento.
LALR(1)	Agrupar estados LR(1) compatíveis.	Muito comum em geradores de parsers.

Para a disciplina, o foco é compreender o mecanismo shift-reduce e a leitura de tabelas. A implementação do parser da MINIC pode usar uma tabela construída manualmente para um subconjunto da gramática ou uma ferramenta geradora, desde que a equipe documente a gramática, as ações e os testes.

Boa prática: não esconda conflitos. Registre o estado, o token que provoca o conflito, as ações candidatas e a decisão adotada.

8. Aplicação à linguagem MINIC

O parser da MINIC recebe os tokens produzidos na Etapa 1 e deve construir a AST da Etapa 2. Uma gramática inicial pode incluir declarações, atribuições, expressões, chamadas e retorno:

```
programa → função
função → INT MAIN ( ) bloco
bloco → { lista_comandos }
comando → declaração ; | atribuição ; | chamada ; | RETURN expressão ;
expressão → expressão + termo | termo
termo → termo * fator | fator
fator → ( expressão ) | ID | NUM
```

Durante as reduções, associe ações semânticas às produções. Por exemplo, ao reduzir **expressão** → **expressão + termo**, crie um nó AST de adição; ao reduzir **fator** → **NUM**, crie um nó literal.

Redução	Ação para a AST
fator → NUM	Criar Literal(valor).
fator → ID	Criar Identifier(nome).
expressão → expressão + termo	Criar Binary(PLUS, esquerda, direita).
comando → ID = expressão ;	Criar Assignment(nome, expressão).

9. Tratamento de erros sintáticos

Erros devem ser relatados com posição, token atual e uma expectativa útil. Uma mensagem como “erro de sintaxe” é insuficiente para depuração.

Exemplo:

```
Entrada: int x = 2 + ;  
Mensagem: linha 1, coluna 13: expressão esperada após "+"; encontrado ";".
```

Estratégias comuns:

- **Modo pânico:** descartar tokens até encontrar um sincronizador, como “;” ou “}”.
- **Recuperação local:** inserir ou remover um token quando a correção for inequívoca.
- **Produções de erro:** incluir regras específicas para orientar a recuperação.

O parser deve evitar entrar em loop após um erro. A recuperação é parte da qualidade do compilador: permite encontrar mais de um problema em uma única execução, sem mascarar a causa original.

Checklist de diagnóstico: estado atual • token de lookahead • pilha • célula ACTION consultada • produção escolhida • posição no código-fonte.

10. Atividade prática — parser shift-reduce

Parte A — simulação. Use a gramática de expressões da aula e registre, para cada passo, a pilha, a entrada restante e a ação. Teste: `id + id * id`.

Parte B — tabela. Identifique as células ACTION/GOTO necessárias para reconhecer declarações e atribuições simples da MINIC. Marque células ainda não definidas.

Parte C — implementação. Escreva um laço de parser que: (1) leia o estado no topo da pilha; (2) consulte o token atual; (3) execute a ação; (4) atualize a AST ou relate o erro.

Parte D — testes. Inclua casos válidos e inválidos:

Categoria	Exemplo
Válido	<code>int x = 2 + 3;</code>
Precedência	<code>x = 2 + 3 * 4;</code>
Parênteses	<code>x = (2 + 3) * 4;</code>
Token ausente	<code>x = 2 + ;</code>
Delimitador ausente	<code>int x = 2</code>
Uso do parser	<code>print(x);</code>

Entrega parcial da Etapa 2: gramática atualizada, decisões de precedência, tabela ou especificação do parser, AST produzida e conjunto de testes.

11. Exercícios de fixação

1. Explique a diferença entre deslocar e reduzir. Em que momento cada ação é apropriada?
2. Para a gramática $E \rightarrow E + T \mid T$ e a entrada `id + id`, descreva uma sequência possível de ações shift-reduce.
3. O que caracteriza um item completo? Qual ação ele sugere?
4. Diferencie conflito shift/reduce de conflito reduce/reduce.
5. Por que a tabela GOTO é consultada depois de uma redução?
6. Proponha uma mensagem de erro para a entrada `print( ;`.
7. Explique como uma redução pode criar um nó da AST.

Desafio. Estenda a gramática para incluir o operador relacional `<` sem alterar a precedência de `+` e `*`. Indique onde podem surgir conflitos e como resolvê-los.

12. Síntese e referências

Ideia central. A análise sintática ascendente reconhece a entrada de baixo para cima. A pilha registra o contexto, ACTION decide o próximo passo e GOTO conduz as reduções entre estados. Em parsers LR, os estados representam conjuntos de situações possíveis dentro da gramática.

Próxima aula. A Aula 9 aprofunda a construção e o uso de árvores sintáticas abstratas, conectando as reduções do parser à representação que será usada na análise semântica.

Referência principal

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson. Capítulo 4 — Análise sintática, especialmente as seções sobre análise ascendente, parsing LR, itens, tabelas e tratamento de erros.

Observação sobre as figuras

As figuras desta apostila são esquemas didáticos originais, elaborados para representar os conceitos do capítulo indicado e preservar a continuidade visual das apostilas da disciplina. Quando a edição impressa estiver disponível, recomenda-se inserir junto às figuras a página exata correspondente da edição utilizada.

MINIC • Etapa 2 — Parser e AST

Produto esperado: um parser capaz de reconhecer o subconjunto definido da MINIC, produzir uma AST consistente e apresentar mensagens de erro localizadas.